MLDS Assignment 1

GitHub username: Dorrie-ge
Repository: mlds-assignment1-rl
https://github.com/Dorrie-ge/mlds-assignment1-rl

Measured experiments on macOS ARM, CPU, Python 3.12; one PyTorch thread. The repository is private: course staff need access to inspect it. This report records actual runs, including failures. Pong uses a limited training budget and is not claimed to be solved.

Experimental summary

Environment	Baseline	Train episodes	Eval episodes	Mean score	Sample SD
CartPole-v1	none	2000	500	9.370	0.755
CartPole-v1	value	2000	500	170.168	11.184
ALE/Pong-v5	none	200	500	-21.000	0.000
ALE/Pong-v5	value	200	500	-21.000	0.000

All four main runs use actor learning rate 0.01 and seed 0. Each policy is frozen and sampled stochastically in 500 evaluation episodes. Evaluation scores are undiscounted game rewards. Standard deviations use ddof=1. Training-seed uncertainty is distinct from evaluation variability.

How to reproduce

Install requirements-lock.txt in a Python 3.12 virtual environment. Run shuttle.py; then policy_gradient.py with --game cartpole/pong, --baseline none/value, --episodes 2000 for CartPole or 200 for Pong, --eval-episodes 500, and a separate --out directory. The exact commands and supplementary learning-rate trials are in README.md. Run validate_policy_gradient.py against each baseline checkpoint, then python build_report.py.

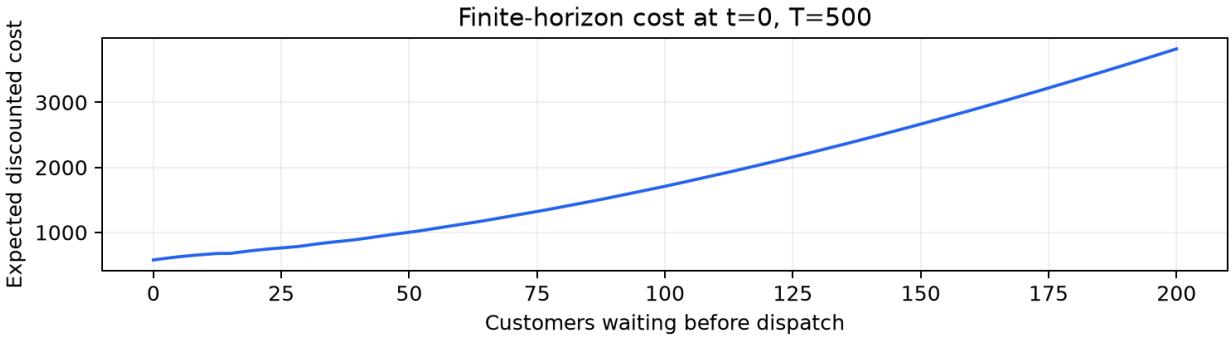
Objectives and baseline

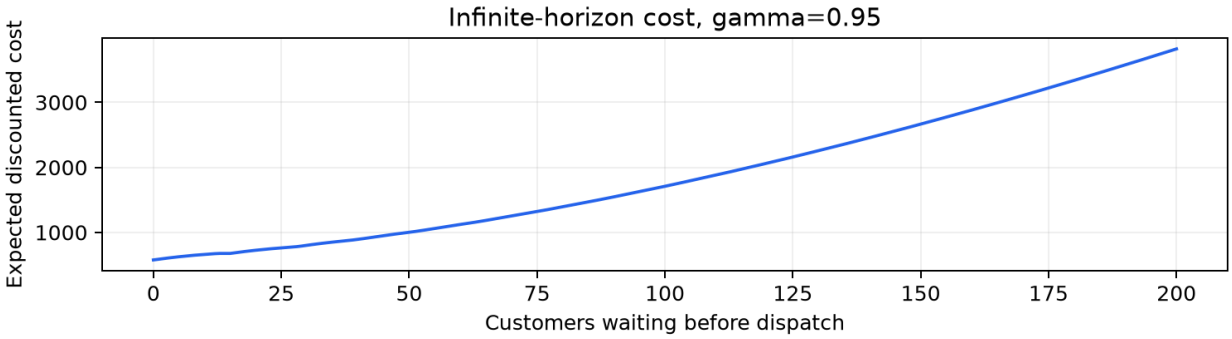
The shuttle objective minimizes expected discounted cost. Game policies maximize E[sum_t gamma^t r_t]. Define G_t=sum_{k=t}^{T-1} gamma^(k-t) r_k. The actor loss is -sum_t gamma^t log pi(a_t|s_t)(G_t-b(s_t)). A separate value network predicts b(s_t); its prediction is detached and obtained before fitting the current trajectory. It uses Monte Carlo MSE and Adam with learning rate 0.001.

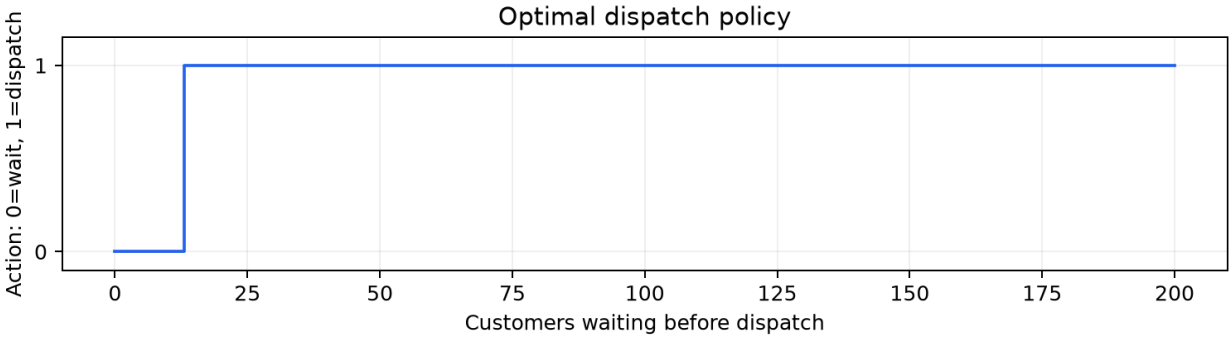
1. Shuttle dispatch: model and solutions

State s is the pre-dispatch queue, 0 through 200; action a is 0 (wait) or 1 (dispatch one shuttle). Let l=max(s-15a,0). Cost is 100a+2l; next state is min(200,l+A), with independent uniform A in {1,2,3,4,5}. Dispatch precedes holding charges and arrivals. Overflow is discarded without penalty; empty dispatch still costs 100. Ties choose wait.

Finite-horizon backward induction sets J_500=0 and enumerates all state-action-arrival outcomes in J_t(s)=min_a[c(s,a)+0.95 E J_{t+1}(s')]. Enumeration is interpreted as this feedback-policy dynamic program. VI repeats the infinite-horizon Bellman update; PI solves (I-0.95 P_pi)J=c_pi before greedy improvement. Values below are positive costs, not reward-maximizing values; use V=-J if rewards are -cost.





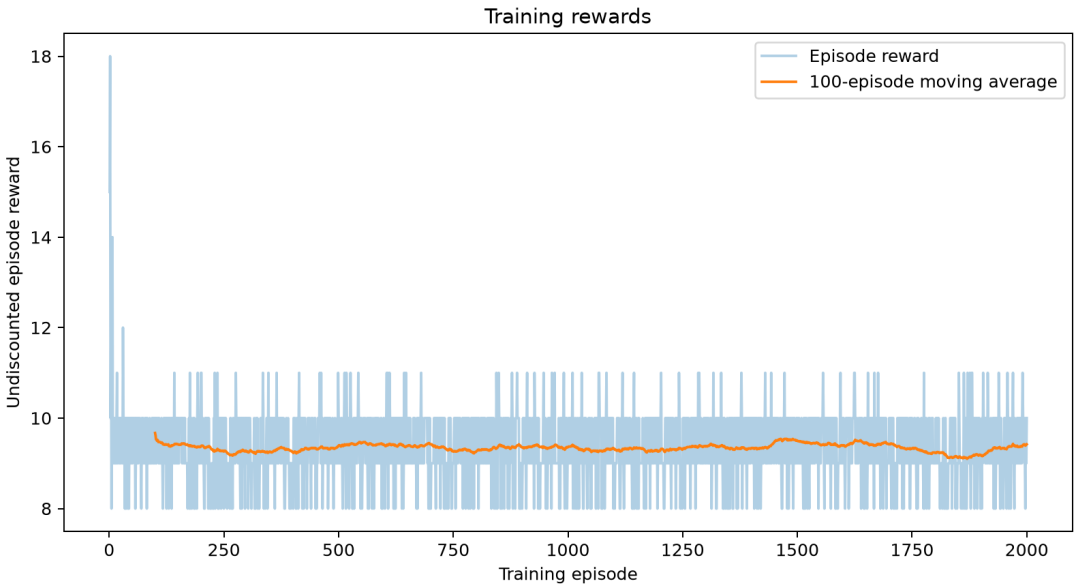


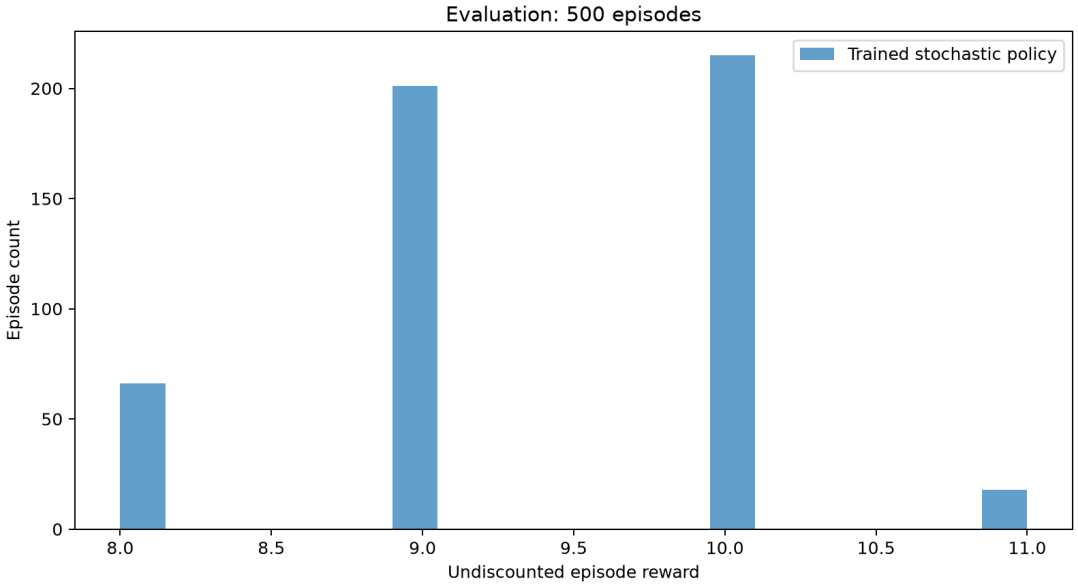
The optimal infinite-horizon policy dispatches at s >= 13. J*(0)=576.670168. VI took 518 iterations and PI 6. A threshold below capacity reflects the cost of waiting. Finite-horizon actions near the terminal time need not match the infinite-horizon policy.

2. CartPole-v1: baseline=none

Training: 2000 episodes, seed 0, gamma=0.95, actor Adam lr=0.01. Final evaluation: 500 episodes; mean=9.370, sample SD=0.755. Random-policy mean=21.906, sample SD=11.218.

The policy maps the four observed coordinates through 128 tanh units to two logits. Both terminated and truncated end a rollout. No value bootstrap is added at truncation; this optimizes the sampled finite episode. Training and evaluation seeds are disjoint.





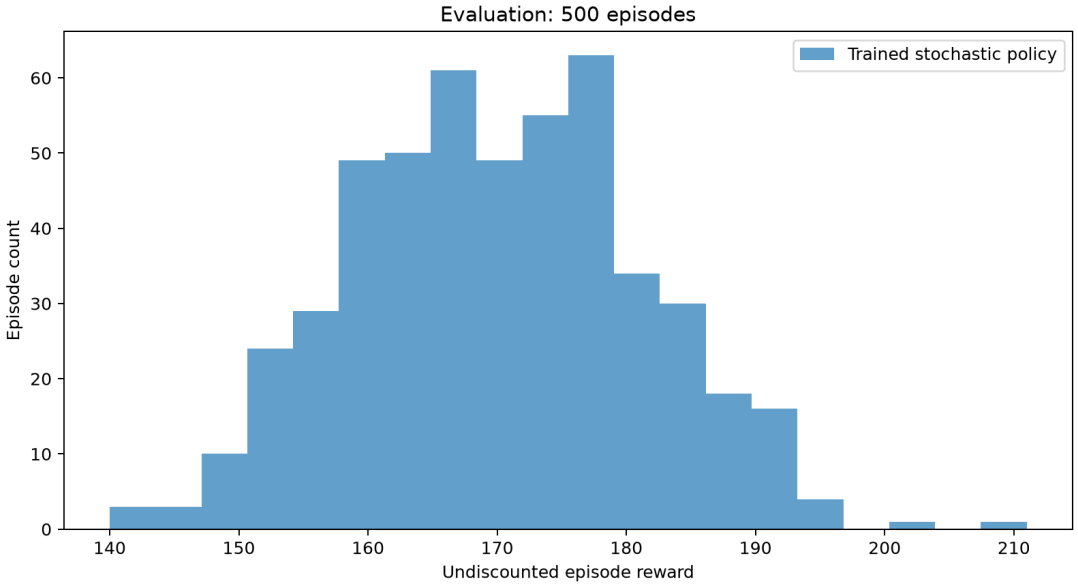
This lr=0.01 run collapsed below the random reference. It is retained as a failure. The supplementary smaller-learning-rate runs test stability; results appear later.

3. CartPole-v1: baseline=value

Training: 2000 episodes, seed 0, gamma=0.95, actor Adam lr=0.01. Final evaluation: 500 episodes; mean=170.168, sample SD=11.184. Random-policy mean=21.906, sample SD=11.218.

The policy maps the four observed coordinates through 128 tanh units to two logits. Both terminated and truncated end a rollout. No value bootstrap is added at truncation; this optimizes the sampled finite episode. Training and evaluation seeds are disjoint.



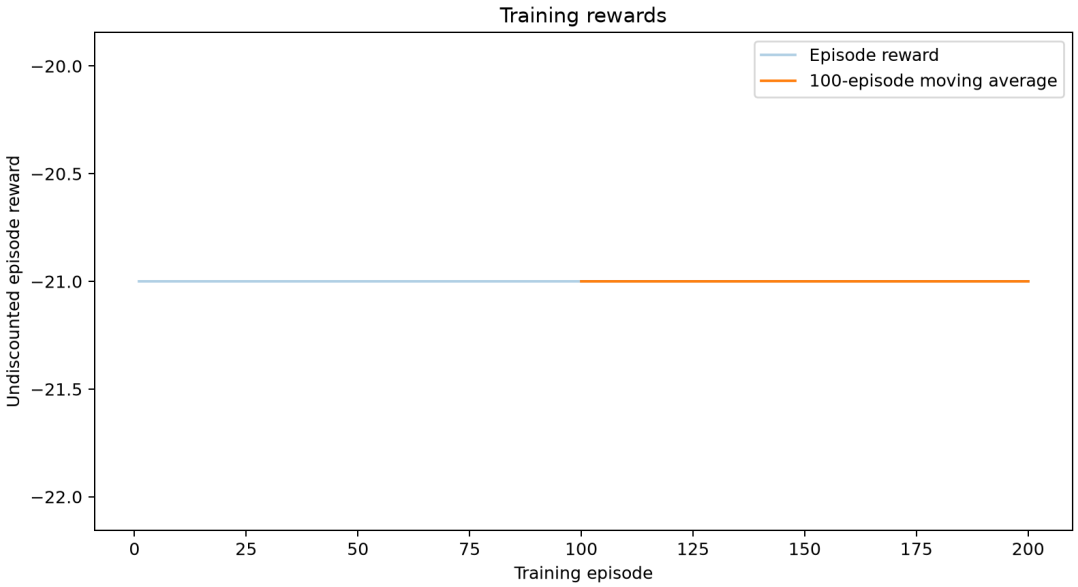


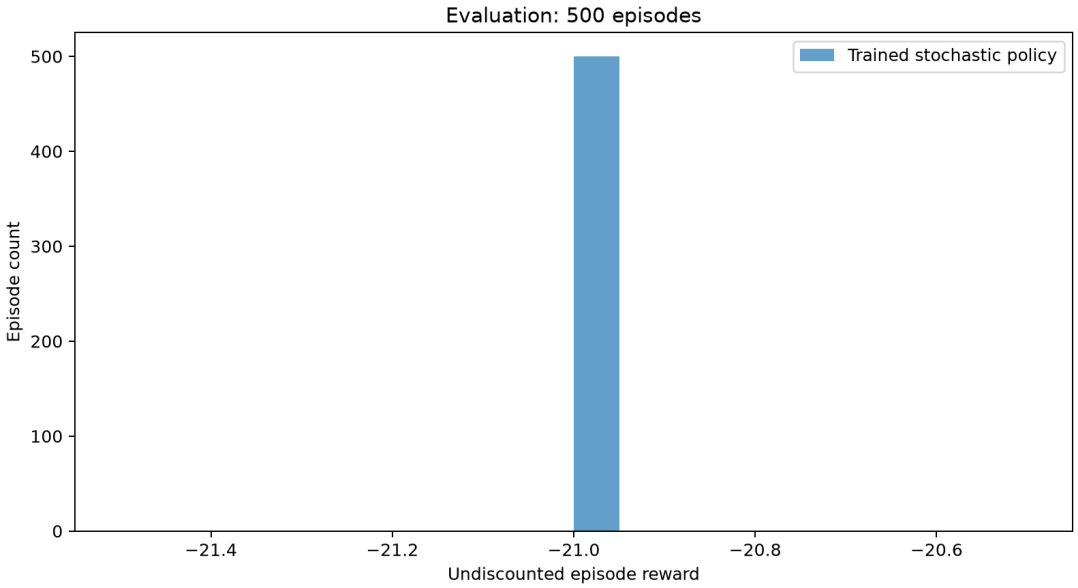
This run exceeds the random reference but does not establish mastery of the 500-step task. Reward improvements alone do not establish that gradient variance was reduced.

4. ALE/Pong-v5: baseline=none

Training: 200 episodes, seed 0, gamma=0.99, actor Adam lr=0.01. Final evaluation: 500 episodes; mean=-21.000, sample SD=0.000. Random-policy mean=-20.412, sample SD=0.829.

The environment is ALE/Pong-v5 with its default frameskip=4 and sticky-action probability 0.25. Two policy logits map only to actions 2 and 3. Inputs are consecutive binary 80x80 frames; history resets every episode. The CNN has 16 filters (8x8, stride 4), then 32 filters (4x4, stride 2), then 128 hidden units. There are no point-based return resets.



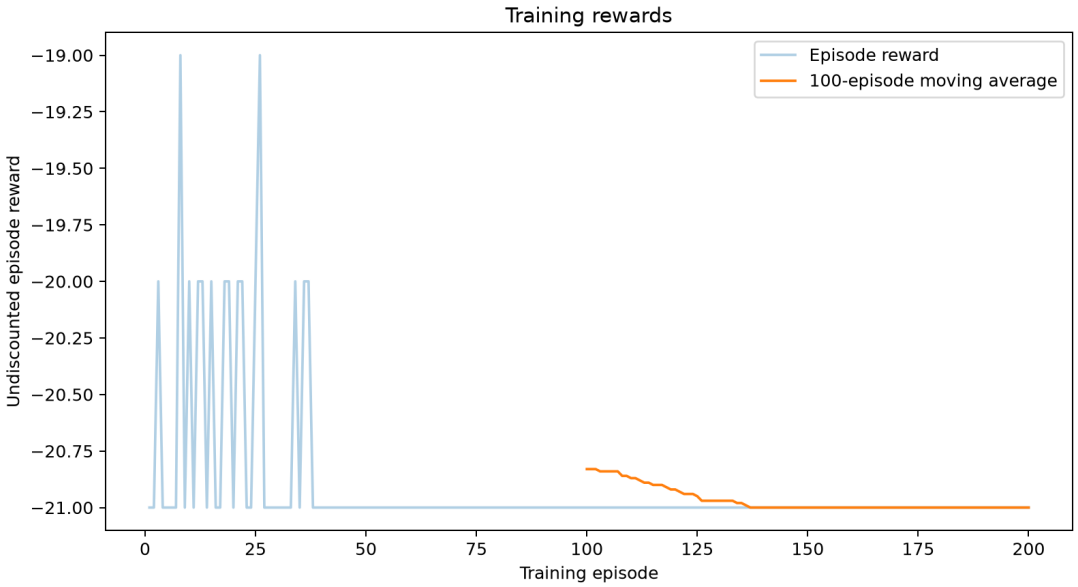


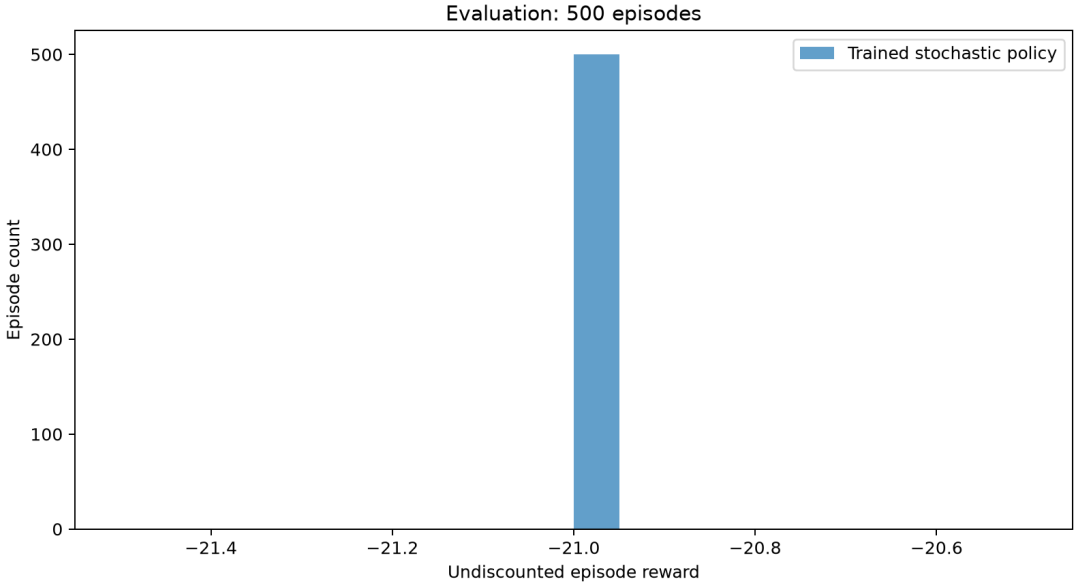
This short-budget run does not establish successful Pong play. Sparse rewards and the outer gamma^t weighting make early training difficult. Reported poor scores are measured results; no longer-run performance is inferred from them.

5. ALE/Pong-v5: baseline=value

Training: 200 episodes, seed 0, gamma=0.99, actor Adam lr=0.01. Final evaluation: 500 episodes; mean=-21.000, sample SD=0.000. Random-policy mean=-20.412, sample SD=0.829.

The environment is ALE/Pong-v5 with its default frameskip=4 and sticky-action probability 0.25. Two policy logits map only to actions 2 and 3. Inputs are consecutive binary 80x80 frames; history resets every episode. The CNN has 16 filters (8x8, stride 4), then 32 filters (4x4, stride 2), then 128 hidden units. There are no point-based return resets.





This short-budget run does not establish successful Pong play. Sparse rewards and the outer gamma^t weighting make early training difficult. Reported poor scores are measured results; no longer-run performance is inferred from them.

6. Validation and edge cases

Independent validation of the shuttle algorithm

Experiment	Observed evidence
Tiny problem: 5 states, 2 arrivals, 4 steps	Independent recursive tree enumeration vs backward induction: max error 0
VI vs PI	Max value difference 1.965e-09; policy mismatches 0
PI Bellman optimality residual	9.095e-13
Finite T=500 vs infinite horizon	Max value difference 2.979e-09
Free dispatch; zero capacity	All positive states dispatch when free; never dispatch with zero capacity
Zero discount; upper queue boundary	Values equal minimum immediate costs; s=200 transitions verified

At s=0, dispatch costs 100 and never creates negative passengers. At s=200, waiting clips all next states to 200; dispatch gives 186 through 190. The finite-horizon terminal value is explicitly zero. The parameter experiments test qualitative predictions of the model, rather than only input formats or absence of runtime errors.

Independent Monte Carlo cost check

Initial state	DP cost	Simulation mean	Standard error
0	576.670	576.769	0.156
12	672.892	673.024	0.153
13	676.670	676.848	0.157
15	676.670	676.286	0.157
200	3821.586	3821.739	0.375

Each initial state uses 40,000 independent 500-step simulations with seed 314159. Four of five DP values are inside their marginal 95% intervals; the state-15 discrepancy is 2.44 standard errors. All five are inside simultaneous 95% Bonferroni intervals (approximately mean +/- 2.576 SE). The maximum omitted discounted tail is below 6.9e-8. The state-15 result is retained rather than rerunning seeds to force agreement.

6. Validation (continued)

Analytic and numerical policy-gradient checks

An independent two-step action enumeration compares the exact discounted objective derivative with the expected score-function gradient, with and without a constant baseline. Both absolute errors are below 6e-8. For the actual CartPole actor, one parameter is also checked using a central finite difference; the absolute discrepancy is about 4.2e-12.

Direct measurement of gradient variance

Frozen policy	Trajectories	Var trace: plain	Var trace: baseline	Ratio
cartpole	64	7281	6.768	0.00092955
pong	16	209.8	34.04	0.1622

cartpole: paired bootstrap 95% percentile interval for baseline/plain variance is [0.00070475, 0.00127]. The actor and critic were frozen at training episode 2000; seeds start at 8,000,000.

pong: paired bootstrap 95% percentile interval for baseline/plain variance is [0.11538, 0.21726]. The actor and critic were frozen at training episode 20; seeds start at 8,000,000.

Variance is the trace of the sample covariance of the full actor-gradient vector, computed on the same held-out trajectories for both estimators. The interval uses 300 paired bootstrap resamples. A ratio below 1 indicates empirical reduction at this checkpoint only. Small samples and a learned baseline do not justify a universal claim that baselines improve every task.

The final Pong actor was almost deterministic: gradient covariance at episode 200 was approximately 4e-49, so its variance ratio is numerically uninformative. That diagnostic is retained separately. The meaningful Pong comparison above instead uses a supplementary 20-episode checkpoint trained with lr=0.0001 and seed 7. Its 2-episode score evaluation is only a smoke test, not a replacement for the main 500-episode evaluation.

7. Stability experiments and limitations

Baseline	Learning rate	Seed	Train episodes	Eval mean	Eval SD
none	0.01	0	2000	9.370	0.755
none	0.001	0	2000	300.240	97.968
none	0.001	1	2000	92.460	31.626
none	0.001	2	2000	106.554	28.962
none	0.003	0	2000	171.432	42.960
value	0.01	0	2000	170.168	11.184
value	0.001	0	2000	220.218	45.618
value	0.001	1	2000	362.986	121.367
value	0.001	2	2000	214.266	69.583

Smaller learning rates are a stability intervention, not a change in the task. The table retains every run, including lr=0.01 collapse. Three matched seeds at lr=0.001 provide an initial sensitivity check. Final evaluation means from hyperparameter exploration should not be interpreted as performance on an untouched model-selection test set.

Evaluation variability describes repeated rollouts of one learned policy; training-seed variability describes the learning procedure. Neither a narrow rollout histogram nor a single successful seed proves robust learning. Pong has one seed and only 200 training episodes per configuration. Its results are limited-budget demonstrations, not completed high-performance training.

8. Three additional months

For shuttle dispatch, test larger queue caps and alternative overflow rules, nonstationary or heavy-tailed arrivals, different capacity and cost ratios, and sensitivity of the threshold. Independently simulate the resulting policies to compare Monte Carlo costs and DP predictions.

For policy gradients, increase independent seeds and budgets; compare constant and value baselines, learning rates, batch sizes, network sizes, and frame representations under matched budgets. Repeat frozen-policy gradient-variance measurements at early, middle, and late checkpoints. Study sparse rewards and the effect of the exact start-state discounted objective. Label alternative objectives or environment modifications explicitly. Evaluate robustness to initial conditions, observation noise, and sticky-action probability in separate experiments.

Sources and reproducibility

Assignment source: mlds_assignment1.pdf, supplied course handout, pages 1-4. Environment reference: https://ale.farama.org/environments/pong/. Algorithm reference: OpenAI Spinning Up, Intro to Policy Optimization, https://spinningup.openai.com/en/latest/spinningup/rl_intro3.html. Raw CSVs, JSON summaries, model weights, exact dependency versions, and commands are in the repository. AI assistance was used to develop code, debugging, and documentation; all numeric experiment results here were generated by actual execution.